



UNIVERSIDAD ESTATAL AMAZÓNICA

PRACTICA N°

1

TIPO DE PRÁCTICA:

PRÁCTICAS EXPERIMENTALES DE ESCRITORIO

MATERIA:

ESTRUCTURA DE DATOS

PARALELO:

E

ESTUDIANTE/S:

MINAYA GARCIA JEFFERSON CRISTOPHER

DOCENTE:

SANTIAGO ISRAEL NOGALES GUERRERO

PERIODO

2026 – 2026

1. Introducción

En el panorama actual del desarrollo de software, la Programación Orientada a Objetos (POO) y las estructuras de datos abstractas se consolidan como los componentes fundamentales para la construcción de sistemas robustos, escalables y mantenibles. Este informe detalla el diseño conceptual y la implementación práctica de un sistema de gestión de información clínica, enfocado en evaluar la eficiencia de las estructuras lineales (como vectores y registros) en la resolución de problemas organizacionales del mundo real.

Fundamentación Teórica

El manejo óptimo de los recursos de memoria y la velocidad de procesamiento dependen directamente de cómo se estructuran los datos dentro de un sistema informático. Bajo la perspectiva de Joyanes Aguilar (2020), un registro se define como una estructura de datos heterogénea que permite unificar múltiples variables de distintos tipos bajo un identificador común. Esta propiedad resulta idónea para modelar entidades complejas de la realidad, tales como "Pacientes" o "Turnos Médicos", donde coexisten datos de texto, numéricos y cronológicos.

Complementariamente, los vectores (abstraídos en el lenguaje Python mediante el uso de listas dinámicas) actúan como contenedores secuenciales. Estas colecciones permiten indexar, almacenar y manipular conjuntos de objetos de forma ordenada, facilitando operaciones cíclicas e iteraciones en la memoria volátil del ordenador.

Justificación

La automatización del agendamiento y control de turnos en un entorno médico justifica plenamente la convergencia de estos enfoques tecnológicos. Una clínica demanda la manipulación precisa de información diversa y vinculada entre sí (identificaciones, horarios, especialidades facultativas). Al implementar una arquitectura basada en POO y estructuras lineales, se garantiza una organización rigurosa de los flujos de datos, sentando las bases operativas para ejecutar búsquedas selectivas, ordenamientos sistemáticos y visualizaciones estructuradas de cara al usuario final.

2. Desarrollo

Para materializar los conceptos teóricos expuestos, se seleccionó y analizó el siguiente caso de estudio: Sistema Automatizado de Agenda de Turnos y Gestión de Pacientes para una Clínica Médica.

Procedimiento Aplicado

Análisis de Requerimientos: Se realizó un levantamiento funcional para determinar las necesidades críticas del entorno hospitalario. Se estableció como prioridad la captura validada de los datos demográficos del paciente y la posterior asignación de citas vinculadas a ramas médicas específicas.

Diseño de Clases y Modelado: Se estructuró el software bajo el principio de responsabilidad única. Se diseñó la clase *Paciente* para operar como un registro contenedor de atributos personales, y la clase *Turno*, cuya función es establecer una asociación o relación de agregación entre un paciente específico y los metadatos de su cita (fecha, hora y especialidad).

Implementación de la Lógica de Negocio: La manipulación de los flujos informáticos se concentró en una clase controladora denominada *GestionClinica*. Esta entidad administra colecciones dinámicas (*self.pacientes* y *self.turnos*) mediante arreglos unidimensionales (listas de Python) que operan en memoria RAM.

Despliegue de la Interfaz de Usuario: Con el objetivo de garantizar la usabilidad del sistema sin añadir sobrecarga computacional innecesaria, se programó una interfaz de línea de comandos (CLI). Este menú interactivo por consola guía al operador en las tareas de inserción, validación y lectura de registros.

Análisis Detallado de las Estructuras de Datos Utilizadas

Estructura	Implementación en Código	Justificación Técnica	Ventajas Principales	Limitaciones Identificadas
Registro (Estructura de Datos Abstraída)	Clases <i>Paciente</i> y <i>Turno</i>	Permite el encapsulamiento de atributos heterogéneos bajo una sola entidad lógica, protegiendo el estado de los datos.	Modularidad del código, alta legibilidad, facilidad de mantenimiento y reutilización de componentes.	Presenta un consumo de memoria ligeramente superior en comparación con el uso aislado de tipos primitivos debido a los metadatos de los objetos.
Vector (Arreglo Lineal / Lista Dinámica)	Listas en memoria: <i>self.pacientes</i> y <i>self.turnos</i>	Proporciona un mecanismo de almacenamiento contiguo y secuencial para colecciones de objetos complejos.	Acceso inmediato a elementos mediante índices directos ($O(1)$) y alta flexibilidad para expandirse dinámicamente.	Deficiencia en el rendimiento ante la escalabilidad; las búsquedas e inserciones masivas requieren un recorrido lineal ($O(n)$), ralentizando el sistema con grandes volúmenes de datos.

3. Resultados y Evidencias

El entorno informático desarrollado responde con éxito a los estímulos del operador, ejecutando transacciones lógicas en tiempo real que cumplen con las directrices de diseño. A través de las trazas de ejecución en consola, se validaron los siguientes hitos operativos:

Persistencia temporal y validación: Registro e indexación de pacientes nuevos asegurando el formato correcto de los campos.

Asociación relacional: Vinculación inequívoca entre la identidad de un paciente y un turno médico disponible.

Formateo de salida: Despliegue de la información de manera organizada mediante estructuras tabulares limpias.



Analítica básica: Procesamiento elemental de datos para reflejar métricas globales sobre el uso de la agenda.

Evidencia de Ejecución en Consola

LISTADO DE TURNOS AGENDADOS

FECHA	HORA	ESPECIALIDAD	PACIENTE
2026-06-20	09:00	CARDIOLOGÍA	JUAN PÉREZ
2026-06-21	10:30	ODONTOLOGÍA	MARIA LOPEZ

```
PS C:\Users\Minaya\Documents\UEA\SEMESTRE 3> & C:/Users/Minaya/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Minaya/Documents/UEA/SEMESTRE 3/ESTRUCTURA DE DATOS/S4/CODE/ubuntu/agenda_clinica.py"

=====
SISTEMA DE GESTIÓN DE TURNOS - CLÍNICA
=====
1. Registrar Paciente
2. Agendar Turno
3. Ver Lista de Turnos
4. Ver Estadísticas
5. Salir

Seleccione una opción: 5
Saliendo del sistema...

PS C:\Users\Minaya\Documents\UEA\SEMESTRE 3> & C:/Users/Minaya/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Minaya/Documents/UEA/SEMESTRE 3/ESTRUCTURA DE DATOS/S4/CODE/ubuntu/agenda_clinica.py"

=====
SISTEMA DE GESTIÓN DE TURNOS - CLÍNICA
=====
1. Registrar Paciente
2. Agendar Turno
3. Ver Lista de Turnos
4. Ver Estadísticas
5. Salir

Seleccione una opción: 1
--- REGISTRO DE PACIENTE ---
Cédula: 0912345678
Nombre: Maria
Apellido: Lopez
Edad: 25
Teléfono: 0987654321
¡Paciente registrado con éxito!

Seleccione una opción: 2
--- AGENDAR TURNO ---
Cédula del paciente: 0912345678
Fecha (AAAA-MM-DD): 2026-06-21
Hora (HH:MM): 10:30
Especialidad: Odontología
¡Turno agendado con éxito!

Seleccione una opción: 3
--- LISTADO DE TURNOS AGENDADOS ---
FECHA      | HORA   | ESPECIALIDAD | PACIENTE
-----
2026-06-20 | 09:00  | Cardiología  | Juan Pérez
2026-06-21 | 10:30  | Odontología  | Maria Lopez

Seleccione una opción: 4
Total de pacientes registrados: 2
Total de turnos agendados: 2

Seleccione una opción: 5
Saliendo del sistema...
```

4. Conclusiones

La adopción del paradigma de la Programación Orientada a Objetos demostró ser una metodología sumamente eficaz para abstraer problemáticas complejas del plano real (procesos clínicos) y transformarlas en modelos de software modulares, legibles e intuitivos.

Si bien los vectores y listas dinámicas ofrecen una solución inmediata, simple y de bajo coste de desarrollo para administrar colecciones de datos a pequeña escala, su rendimiento decae al aumentar el volumen de registros. Para entornos corporativos o de producción masiva, es imperativo migrar hacia estructuras avanzadas como tablas indexadas (Tablas Hash) o motores de bases de datos relacionales de alta disponibilidad.

Se alcanzaron con éxito las metas académicas y técnicas planteadas para esta práctica, consolidando una aplicación funcional e integrada donde convergen la teoría de los tipos de datos abstractos, el encapsulamiento de datos y el flujo lógico guiado por estructuras de control.

5. Bibliografía

- Joyanes Aguilar, L. (2020). Fundamentos de Programación: Algoritmos, Estructuras de Datos y Objetos. 5ta Edición. McGraw-Hill.
- Sedgewick, R., & Wayne, K. (2019). Algorithms. 4th Edition. Addison-Wesley Professional.
- Enlace de repositorio: <https://github.com/MINAYA101/SEMESTRE-3/tree/master/ESTRUCTURA%20DE%20DATOS/S4/CODE/ubuntu>

6. ANEXOS

